

Unit-2: Fundamentals of React.js

2.1 Overview of React

ReactJS is JavaScript library used for building reusable UI components. According to React official documentation, following is the definition –

React is a library for building composable user interfaces. It encourages the creation of reusable UI components, which present data that changes over time. Lots of people use React as the V in MVC. React abstracts away the DOM from you, offering a simpler programming model and better performance. React can also render on the server using Node, and it can power native apps using React Native. React implements one-way reactive data flow, which reduces the boilerplate and is easier to reason about than traditional data binding.

React Features

Currently, ReactJS is gaining quick popularity as the best JavaScript framework among web developers. It is playing an essential role in the front-end ecosystem. The important features of ReactJS are as follows.

JSX

JSX stands for JavaScript XML. It is a JavaScript syntax extension. Its an XML or HTML-like syntax used by ReactJS. It extends the ES6 so that HTML-like text can co-exist with JavaScript react code.

Components

ReactJS is all about components. ReactJS application is made up of multiple components, and each component has its own logic and controls. These components can be reusable which help you to maintain the code when working on larger scale projects.

One-way Data Binding

ReactJS is designed in such a manner that follows unidirectional data flow or one-way data binding. The benefits of one-way data binding give you better control throughout the application. If the data flow is in another direction, then it requires additional features.

One way Data Binding means the data can be passed from Parent component to Child component. But to pass the data from Child to Parent, we have to use some additional features.

Virtual DOM

A virtual DOM object is a representation of the original DOM object. It works like a one-way data binding. Whenever any modifications happen in the web application, the entire UI is re-rendered in virtual DOM representation. Then it checks the difference between the previous DOM representation and new DOM. Once it has done, the real DOM will update only the things that have actually changed. This makes the application faster, and there is no wastage of memory.

Simplicity

ReactJS uses JSX file which makes the application simple and to code as well as understand. We know that ReactJS is a component-based approach which makes the code reusable as your need. This makes it simple to use and learn.

Performance

ReactJS is known to be a great performer. This feature makes it much better than other frameworks out there today. The reason behind this is that it manages a virtual DOM. The DOM is a cross-platform and programming API which deals with HTML, XML or XHTML. The DOM exists entirely in memory. Due to this, when we create a component, we did not write directly to the DOM. Instead, we are writing virtual components that will turn into the DOM leading to smoother and faster performance.

Advantage of ReactJS

1. Easy to Learn and USe

ReactJS is much easier to learn and use. Any developer who comes from a JavaScript background can easily understand and start creating web apps using React in a few days. It is the V(view part) in the MVC (Model-View-Controller) model, and referred to as one of the JavaScript frameworks. It is not fully featured but has the advantage of open-source JavaScript User Interface(UI) library, which helps to execute the task in a better manner.

2. Creating Dynamic Web Applications Becomes Easier

To create a dynamic web application specifically with HTML strings was tricky because it requires a complex coding, but React JS solved that issue and makes it easier. It provides less coding and gives more functionality. It makes use of the JSX(JavaScript Extension), which is a particular syntax letting HTML quotes and HTML tag syntax to render particular subcomponents. It also supports the building of machine-readable codes.

3. Reusable Components

A ReactJS web application is made up of multiple components, and each component has its own logic and controls. These components are responsible for outputting a small, reusable piece of HTML code which can be reused wherever you need them. The reusable code helps to make your apps easier to develop and maintain. These Components can be nested with other components to allow complex applications to be built of simple building blocks. ReactJS uses virtual DOM based mechanism to fill data in HTML DOM. The virtual DOM works fast as it only changes individual DOM elements instead of reloading complete DOM every time.

4. Performance Enhancement

ReactJS improves performance due to virtual DOM. The DOM is a cross-platform and programming API which deals with HTML, XML or XHTML. Most of the developers faced the problem when the DOM was updated, which slowed down the performance of the application. ReactJS solved this problem by introducing virtual DOM. The React Virtual DOM exists entirely in memory and is a representation of the web browser's DOM. Due to this, when we write a React component, we did not write directly to the DOM. Instead, we are writing virtual components that react will turn into the DOM, leading to smoother and faster performance.

5. The Support of Handy Tools

React JS has also gained popularity due to the presence of a handy set of tools. These tools make the task of the developers understandable and easier. The React Developer Tools have been designed as Chrome and Firefox dev extension and allow you to inspect the React component hierarchies in the virtual DOM. It also allows you to select particular components and examine and edit their current props and state.

6. Known to be SEO Friendly

Traditional JavaScript frameworks have an issue in dealing with SEO. The search engines generally having trouble in reading JavaScript-heavy applications.

ReactJS overcomes this problem that helps developers to be easily navigated on various search engines. It is because React.js applications can run on the server, and the virtual DOM will be rendering and returning to the browser as a regular web page.

7. The Benefit of Having JavaScript Library

Today, ReactJS is choosing by most of the web developers. It is because it is offering a very rich JavaScript library. The JavaScript library provides more flexibility to the web developers to choose the way they want.

8. Scope for Testing the Codes

ReactJS applications are extremely easy to test. It offers a scope where the developer can test and debug their codes with the help of native tools.

Disadvantage of ReactJS

1. The high pace of development

The high pace of development has an advantage and disadvantage both. In case of disadvantage, since the environment continually changes so fast, some of the developers not feeling comfortable to relearn the new ways of doing things regularly. It may be hard for them to adopt all these changes with all the continuous updates. They need to be always updated with their skills and learn new ways of doing things.

2. Poor Documentation

It is another cons which are common for constantly updating technologies. React technologies updating and accelerating so fast that there is no time to make proper documentation. To overcome this, developers write instructions on their own with the evolving of new releases and tools in their current projects.

3. View Part

ReactJS Covers only the UI Layers of the app and nothing else. So you still need to choose some other technologies to get a complete tooling set for development in the project.

4. JSX as a barrier

ReactJS uses JSX. It's a syntax extension that allows HTML with JavaScript mixed together. This approach has its own benefits, but some members of the development community consider JSX as a barrier, especially for new developers. Developers complain about its complexity in the learning curve.

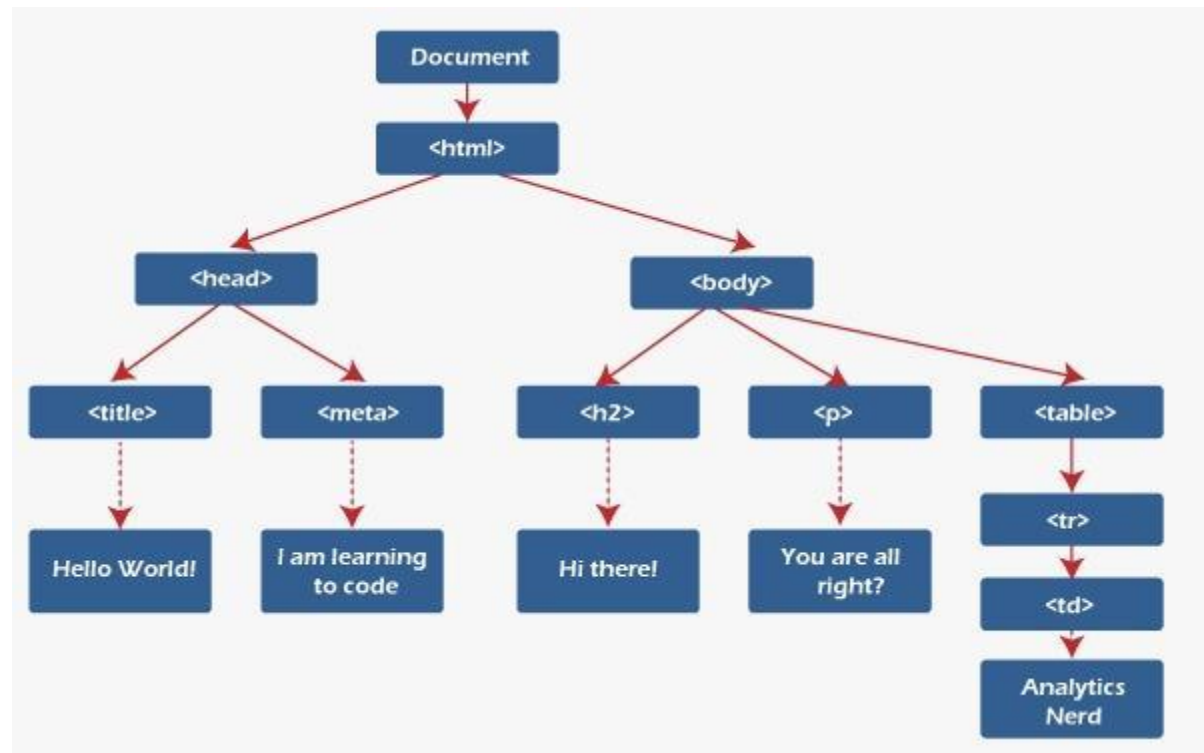
2.1.1 Concepts of React.

React is an open source, JavaScript library for developing user interface (UI) in web application. React is developed and released by Facebook. Facebook is continuously working on the React library and enhancing it by fixing bugs and introducing new features.

If you are using React or learning React, you must have heard of the term “Virtual DOM”. Now what is a Virtual DOM, and why does React use it?

Real DOM

First things first, DOM stands for “Document Object Model”. The DOM in simple words represents the UI of your application. Everytime there is a change in the state of your application UI, the DOM gets updated to represent that change. Now the catch is frequently manipulating the DOM affects performance, making it slow.



What makes DOM manipulation slow?

The DOM is represented as a tree data structure. Because of that, the changes and updates to the DOM are fast. But after the change, the updated element and its children have to be re-rendered to update the application UI. The re-rendering or re-painting of the UI is what makes it slow. Therefore, the more UI components you have, the more expensive the DOM updates could be, since they would need to be re-rendered for every DOM update.

How is Virtual DOM faster?

When new elements are added to the UI, a virtual DOM, which is represented as a tree is created. Each element is a node on this tree. If the state of any of these elements changes, a

new virtual DOM tree is created. This tree is then compared or “diffed” with the previous virtual DOM tree.

Once this is done, the virtual DOM calculates the best possible method to make these changes to the real DOM. This ensures that there are minimal operations on the real DOM. Hence, reducing the performance cost of updating the real DOM.

The image below shows the virtual DOM tree and the diffing process.

The red circles represent the nodes that have changed. These nodes represent the UI elements that have had their state changed. The difference between the previous version of the virtual DOM tree and the current virtual DOM tree is then calculated. The whole parent subtree then gets re-rendered to give the updated UI. This updated tree is then batch updated to the real DOM.

How does React use Virtual DOM

Now that you have a fair understanding of what a Virtual DOM is, and how it can help with performance of your app, lets look into how React leverages the virtual DOM.

In React every UI piece is a component, and each component has a state. React follows the observable pattern and listens for state changes. When the state of a component changes, React updates the virtual DOM tree. Once the virtual DOM has been updated, React then compares the current version of the virtual DOM with the previous version of the virtual DOM. This process is called “diffing”.

Once React knows which virtual DOM objects have changed, then React updates only those objects, in the real DOM. This makes the performance far better when compared to manipulating the real DOM directly. This makes React stand out as a high performance JavaScript library.

In simple words, you tell React what state you want the UI to be in, and it makes sure that the DOM matches that state. The great benefit here is that as a developer, you would not need to know how the attribute manipulation, event handling or the manual DOM updates happen behind the scenes.

All of these details are abstracted away from React developers. All you need to do is update the states of your component as and when needed and React takes care of the rest. This ensures a superior developer experience when using React.

React `render()` function

`render()` is where the UI gets updated and rendered. `render()` is the required lifecycle method in React.

`render()` function is the point of entry where the tree of React elements are created. When a state or prop within the component is updated, the `render()` will return a different tree of React elements. If you use `setState()` within the component, React immediately detects the state change and re-renders the component.

React then figures out how to efficiently update the UI to match the most recent tree changes.

This is when React updates its virtual DOM first and updates only the object that have changed in the real DOM.

Rendering Elements

Elements are the smallest building blocks of React apps.

An element describes what you want to see on the screen:

```
const element = <h1>Hello, world</h1>;
```

Unlike browser DOM elements, React elements are plain objects, and are cheap to create. React DOM takes care of updating the DOM to match the React elements.

Note:

One might confuse elements with a more widely known concept of “components”. We will introduce components in the next section. Elements are what components are “made of”, and we encourage you to read this section before jumping ahead.

Rendering an Element into the DOM

Let’s say there is a `<div>` somewhere in your HTML file:

```
<div id="root"></div>
```

We call this a “root” DOM node because everything inside it will be managed by React DOM.

Applications built with just React usually have a single root DOM node. If you are integrating React into an existing app, you may have as many isolated root DOM nodes as you like.

To render a React element, first pass the DOM element to ReactDOM.createRoot(), then pass the React element to root.render():

```
const root = ReactDOM.createRoot(  
  document.getElementById('root')  
)  
  
const element = <h1>Hello, world</h1>  
root.render(element);
```

It displays “Hello, world” on the page.

Updating the Rendered Element

React elements are immutable. Once you create an element, you can’t change its children or attributes. An element is like a single frame in a movie: it represents the UI at a certain point in time.

The only way to update the UI is to create a new element, and pass it to root.render().

2.1.2 Using React with HTML

Start by including three scripts, the first two let us write React code in our JavaScripts, and the third, Babel, allows us to write JSX syntax and ES6 in older browsers.

Include three CDN's(Content Delivery Network from react.org) in your HTML file:

```
//hello.html  
<!DOCTYPE html>
```



```
<html>

<head>

  <script src="https://unpkg.com/react@18/umd/react.development.js" crossorigin></script>

  <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>

  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>

</head>

<body>


  <div id="mydiv"></div>


  <script type="text/babel">

    function Hello() {

      return <h1>Hello World!</h1>;

    }


    ReactDOM.render(<Hello />, document.getElementById('mydiv'))

  </script>


</body>

</html>
```

Run the React Application

Setting up a React Environment

If you have npx and Node.js installed, you can create a React application by using create-react-app.

If you've previously installed create-react-app globally, it is recommended that you uninstall the package to ensure npx always uses the latest version of create-react-app.

To uninstall, run this command: `npm uninstall -g create-react-app`.

Run this command to create a React application named my-react-app:

```
npx create-react-app my-react-app
```

The create-react-app will set up everything you need to run a React application.

Run the React Application

Now you are ready to run your first real React application!

Run this command to move to the my-react-app directory:

```
cd my-react-app
```

Run this command to run the React application my-react-app:

```
npm start
```

A new browser window will pop up with your newly created React App! If not, open your browser and type localhost:3000 in the address bar.

The result:

Modify the React Application

So far so good, but how do I change the content?

Look in the my-react-app directory, and you will find a src folder. Inside the src folder there is a file called App.js, open it and it will look like this:

```
/myReactApp/src/App.js:
```

```

import logo from './logo.svg';
import './App.css';
function App() {
  return (
    <div className="App">
      <header className="App-header">
        <img src={logo} className="App-logo" alt="logo" />
        <p>
          Edit <code>src/App.js</code> and save to reload.
        </p>
        <a className="App-link" href="https://reactjs.org" target="_blank" rel="noopener
norereferrer">
          Learn React
        </a>
      </header>
    </div>
  );
}

```

export default App;

Try changing the HTML content and save the file.

Example

Replace all the content inside the `<div className="App">` with a `<h1>` element.

See the changes in the browser when you click Save.

```

function App() {
  return (
    <div className="App">

```

```
    <h1>Hello World!</h1>
  </div>
);
}
export default App;
```

The result:

React Folder Structure

When we run the command (npx create-react-app hello-world) in our terminal to create react application the following initial project structure gets generated inside our project folder named hello-world as shown below:

React Folder Structure

.gitignore file is a text file that tells Git which files or folders to ignore in a project.

package.json is a JSON file that holds metadata relevant to the project and it is used for managing the project's dependencies, scripts, version. This file is basically used to give information to npm that allows it to identify the project as well as handle the project's dependencies.

package-lock.json is automatically generated for any operations where npm modifies either the node_modules tree or package.json. It helps while a team is working on private apps to ensure that they are working on the same version of dependencies and sub-dependencies. It also maintains a history of changes done in package.json so, that at any point of time, when required previous changes can be looked back in the package-lock.json file.

node_modules contains all dependencies and sub-dependencies specified in package.json used by React app.

public — This folder contains files that don't require any additional processing by webpack. The index.html file is considered as an entry point for the web application. In the HTML file you can import libraries such as CSS libraries and add meta tags.

src — This folder is the main folder of React application as it contains JavaScript which needs to be processed by webpack. In this folder, you make your components, and then you can import them in app.js. Lastly, src contains registerServiceWorker.js which takes care of caching and updating files for the end-user. It helps in offline capability and faster page loading after the first visit.

2.1.3 React Interactive components: Components within components and Files

A Component is one of the core building blocks of React. In other words, we can say that every application you will develop in React will be made up of pieces called components. Components make the task of building UIs much easier. You can see a UI broken down into multiple individual pieces called components and work on them independently and merge them all in a parent component which will be your final UI.

React Interactive Components: Components within Components &

Files

React components are like functions that return HTML elements. React components let you break up the user interface into separate pieces that can then be reused and handled independently.

A Component is considered as the core building blocks of a React application. It makes the task of building UIs much easier. Each component exists in the same space, but they work independently from one another and merge all in a parent component, which will be the final UI of your application.

There are two types of components:

Class Components

Function Components

Class Components:

A class component must include the extends React.Component statement.

This statement creates an inheritance to React.Component, and gives your component access to React.Component's functions.

The component also requires a render() method, this method returns HTML.

You can pass data from one class to other class components.

You can create a class by defining a class that extends Component and has a render function.

Example:

Create Welcome Component(class)

In App.js file

```
import './App.css';
import Welcome from './Welcome';
function App() {
  return (
    <><Welcome /></>
  );
}
export default App;
```

In Welcome.js file

```
import React,{Component}from "react";
class Welcome extends Component {
  render(){
    return <h1>Inside Welcome class</h1>;
  }
}
export default Welcome;
```

Function Component:

A Function component also returns HTML, and behaves much the same way as a Class component, but Function components can be written using much less code.

The functional component is also known as a stateless component because they do not hold or manage state.

Example: Create Heading Component(function)

In App.js file

```
import './App.css';
import Heading from './Heading';
function App() {
  return (
    <><Heading /></>
  );
}
export default App;
```

In Heading.js file

```
import React from 'react';
function Heading(){
  return <h1> We r in heading function</h1>;
}
export default Heading;
```

Components within Component:

We can refer to components inside other components:

Use the Fybca component inside the Students component:

Example:

```
import React from 'react';
import ReactDOM from 'react-dom/client';

function Fybca() {
  return <h2>I study in FYBCA</h2>;
}

function Students() {
  return (
    <>
    <h1>In which year do you study?</h1>
    <Fybca />
    </>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Students />);
export default Students;
```

Components in Files

React is all about re-using code, and it is recommended to split your components into separate files.

To do that, create a new file with a .js file extension and put the code inside it:

Note that the filename must start with an uppercase character.

First create new file named Car.js

```
function Car() {
  return <h2>Hi, I am a Car!</h2>;
}

export default Car;
```


To be able to use the Car component, you have to import the file in your application.

Import the "Car.js" file in the application, and we can use the Car component as if it was created here.

App.js

```
import "./App.css";
import Car from "./Car";
function App() {
  return (
    <>
    <Car/>
    </>
  );
}
export default App;
```

Props:

Components can be passed as props, which stands for properties.

Props are like function arguments, and you send them into the component as attributes.

Use an attribute to pass a color to the Car component, and use it in the render() function:

Example:Car.jsx

```
function Car(props) {
  return <h2>I am a {props.color} Car!</h2>;
}
export default Car;
```

Props are also how you pass data from one component to another, as parameters.

Send the "brand" property from the Garage component to the Car component:

Example:

```
function Car(props) {  
  return <h2>I am a { props.brand }!</h2>;  
}  
  
function Garage() {  
  return (  
    <>  
      <h1>Who lives in my garage?</h1>  
      <Car brand="Ford" />  
    </>  
  );  
}  
export default Garage;
```

Create a variable named carName and send it to the Car component:

Example

```
function Car(props) {  
  return <h2>I am a { props.brand }!</h2>;  
}  
  
function Garage() {  
  const carName = "Ford";  
  return (  
    <>    <h1>Who lives in my garage?</h1>  
    <Car brand={ carName } />  
  )  
}
```

```
</>
```

```
);}
```

```
export default Garage;
```

React allows you to pass props but only down the family tree. A parent can only pass information to the children. Children can not pass props up to the parent. This is the one way data flow of React.

2.2.2 Conditional statements, Lists

React Conditional Rendering

In React, you can conditionally render components.

There are several ways to do this.

if Statement

We can use the if JavaScript operator to decide which component to render.

Example: Goal.jsx

```
import React from 'react';
```

```
function MissedGoal() {  
  return <h1>MISSED!</h1>;  
}
```

```
function MadeGoal() {  
  return <h1>GOAL!</h1>;  
}
```

```
const Goal=(props)=> {  
  console.log(props);  
  const isGoal = props.isGoal;  
  if (isGoal) {  
    return <MadeGoal/>;  
  }  
  return <MissedGoal/>;  
}  
export default Goal;
```

App.js

```
import './App.css';  
import Goal from './Goal';  
function App() {  
  return (  
    <> <Goal isGoal={true}/> </>  
  );  
}  
export default App;
```

Logical && Operator

Another way to conditionally render a React component is by using the && operator.

Example: Sandwich.jsx

```
import React from 'react';  
function Sandwich(props) {
```

```
const fillings = props.fillings;

return (
  <>
    <h1>Sandwich</h1>
    {fillings.length > 0 &&
      <h2>
        You have {fillings.length} in your sandwich.
      </h2>
    }
  </>
);
}

export default Sandwich;
```

App.jsx

```
import './App.css';
import Sandwich from './Sandwich';

function App() {
  const fillings = ['Cheese', 'Veggies', 'Sauces'];
  return (
    <>
      <Sandwich fillings={fillings} />
    </>
  );
}
```

```
export default App;
```

Ternary Operator

Another way to conditionally render elements is by using a ternary operator.

condition ? true : false

Example: Raining.jsx

```
import React from 'react';
```

```
function NoUmbrella() {
```

```
    return <h1>Don't need Umbrella!</h1>;
```

```
}
```

```
function Umbrella() {
```

```
    return <h1>Need Umbrella!</h1>;
```

```
}
```

```
function Raining(props) {
```

```
    const isRaining = props.isRaining;
```

```
    return (
```

```
        <>{ isRaining ? <Umbrella/> : <NoUmbrella/> }
```

```
        </>
```

```
    );
```

```
}
```

```
export default Raining;
```

App.js

```
import './App.css';
```

```
import Goal from './Goal';
```

```
function App() {  
  return (  
    <> <Raining isRaining={true}/> </>  
  );  
}  
  
export default App;
```

List:

In React, you will render lists with some type of loop.

The JavaScript `map()` array method is generally the preferred method.

Example 1:

```
import React from 'react'  
  
function NameList() {  
  const names=["Bruce","Diana","Clark"]  
  
  return (  
    <div>  
      { names.map(name=> <h2>{name} </h2>)}  
    </div>  
  )  
}  
  
export default NameList;
```

Keys

Keys allow React to keep track of elements. This way, if an item is updated or removed, only that item will be re-rendered instead of the entire list.

Keys need to be unique to each sibling. But they can be duplicated globally.

Example NameListt.js

```
import React from 'react'

function NameListt() {
  const persons=[
    {id:1,name:"Bruce",age:30,skill:"React"},
    {id:2,name:"Diana",age:28,skill:"Angular"},
    {id:3,name:"Clark",age:32,skill:"JavaScript"}
  ]

  const personList=persons.map(person =>
    <h2>
    I am {person.name}.I am {person.age}years old. I know {person.skill}
    </h2>
  )

  return (
    <div>
      {personList}
    </div>
  )
}

export default NameListt;
```


State :

In React, the term "state" refers to an object that holds data that can change over time and influences the behavior and rendering of a component. State is one of the fundamental concepts in React and is used to manage dynamic or mutable data within a component. Here are some key points to understand about state in React:

1. **Stateful Components:** Components that manage and utilize state are often referred to as "stateful" or "class components" because they are typically implemented as JavaScript classes that extend `React.Component`. Functional components can also manage state using React Hooks, introduced in React 16.8 and later.
2. **Initialization:** To use state in a class component, you initialize it in the constructor method of the component by setting `this.state` to an object with the initial data. For functional components, you use the `useState` Hook to initialize state.

Example Class component

```
class Counter extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = { count: 0 };  
  }  
  // ...  
}
```

3. **Updating State:** You should never directly modify the state object; instead, use the `setState` method (class components) or the function returned by the `useState` Hook (functional components) to update state. React will then re-render the component with the updated state, triggering a virtual DOM diffing process to efficiently update the actual DOM.

Example of class component

```
incrementCount = () => {  
  this.setState({ count: this.state.count + 1 });  
}
```

4. **Asynchronous Updates:** State updates in React are asynchronous to improve performance. When you call `setState` or use state update functions, React batches the updates and applies them in the most efficient way. Therefore, you should not rely on the current state value when updating it.
5. **Reactive Rendering:** Whenever state changes, React re-renders the component and its children to reflect the updated data in the user interface. This reactive nature of React makes it easy to build dynamic and interactive user interfaces.
6. **Local to a Component:** State is local to the component that declares it. Other components cannot directly access or modify another component's state. To share data between components, you typically pass data via props or use a state management solution like Redux or the Context API.

Overall, state is a crucial concept in React that enables components to manage and react to changing data, making it a fundamental building block for building dynamic and responsive user interfaces.

props vs state

props

props get passed to the component
Function parameters
props are immutable
props – Functional Components
this.props – Class Components

state

state is managed within the component
Variables declared in the function body
state can be changed
useState Hook – Functional Components
this.state – Class Components

Example:

```
import React, { Component } from 'react'

class Counter extends Component {
  constructor() {
    super()
```

```

    this.state={
      count:10
    }
    inc(){
      //this.state.count=this.state.count+1
      this.setState({
        count: this.state.count-2

        },()=>{console.log('Callback value',this.state.count)})
      console.log(this.state.count)
    }
    render() {
      return (
        <div>
          <h1> count :{this.state.count}</h1>
          <button onClick={()=>this.inc()}>decrement</button>
        </div>
      )
    }
  }
}

export default Counter

```

2.2.3 React Events: Adding events, Passing arguments, Event objects

Adding Events

- React events are written in camelCase syntax: `onClick` instead of `onclick`.
- React event handlers are written inside curly braces: `onClick={shoot}` instead of `onClick="shoot()"`.

React:

```
<button onClick={shoot}>Take the Shot!</button>
```

HTML:

```
<button onclick="shoot()">Take the Shot!</button>
```

Example: Foodstall.jsx

Put the order function inside the Foodstall component:

```
import React from 'react';

function Foodstall() {
  const order = () => {
    alert("Food Ordered ...Your Order is on its way!");
  }
  return (
    <button onClick={order}>Order Food!</button>
  );
}

export default Foodstall;
```

Passing Arguments

To pass an argument to an event handler, use an arrow function.

```
import React from 'react';

function Foodstall() {
  const order = (a) => {
    alert(a);
  }
  return (
    <button onClick={() => order("Pizza!")}>Order Food!</button>
  );
}
```

```
}  
  
export default Foodstall;
```

React Event Object:

Event handlers have access to the React event that triggered the function.

In our example the event is the "click" event.

Example:

Arrow Function: Sending the event object manually:

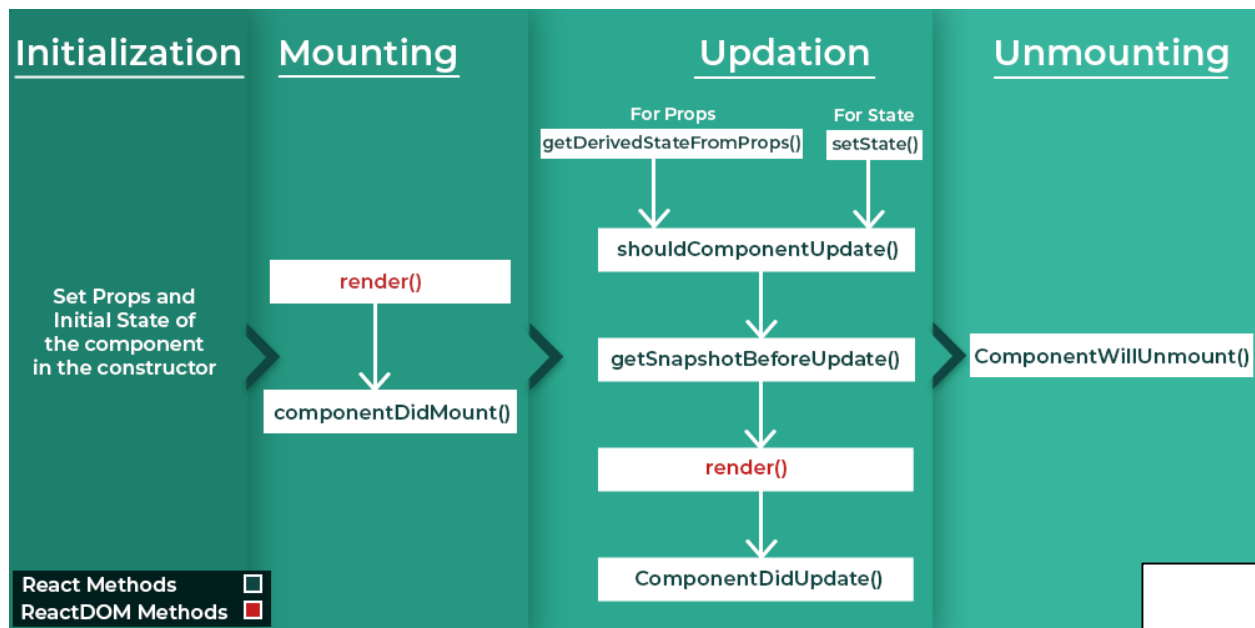
```
import React from 'react';  
  
function Football() {  
  const shoot = (a, b) => {  
    alert(b.type);  
    /*'b' represents the React event that triggered the function.  
    In this case, the 'click' event*/  
  }  
  return (  
    <button onClick={{(event) => shoot("Goal!", event)}}>Take the shot!</button>  
  );  
}  
  
export default Football;
```

4.6 Life Cycle of components:

Every React Component has a lifecycle of its own, lifecycle of a component can be defined as the series of methods that are invoked in different stages of the component's existence. The definition is pretty straightforward but what do we mean by different stages? A React Component can go through four stages of its life as follows.

- **Initialization:** This is the stage where the component is constructed with the given Props and default state. This is done in the constructor of a Component Class.
- **Mounting:** Mounting is the stage of rendering the JSX returned by the render method itself.
- **Updating:** Updating is the stage when the state of a component is updated and the application is repainted.
- **Unmounting:** As the name suggests Unmounting is the final step of the component lifecycle where the component is removed from the page.

React provides the developers with a set of predefined functions that if present are invoked around specific events in the lifetime of the component. Developers are supposed to override the functions with desired logic to execute accordingly. We have illustrated the gist in the following diagram.



Functions of each Phase of Lifecycle:

Initialization

In this phase, the developer has to define the props and initial state of the component. This is generally done in the constructor of the component. The following code snippet describes the initialization process.

```
class Clock extends React.Component {
  constructor(props)
  {
    // Calling the constructor of
    // Parent Class React.Component
    super(props);

    // Setting the initial state
    this.state = { date : new Date() };
  }
}
```

Mounting

Mounting is the phase of the component lifecycle when the initialization of the component is completed and the component is mounted on the [DOM](#) and rendered for the first time on the webpage. Now React follows a default procedure in the Naming Conventions of these predefined functions where the functions containing “Will” represents before some specific phase and “Did” represents after the completion of that

phase. The mounting phase consists of two such predefined functions as described below.

- First initialize the data and the states in the constructor
- [**componentDidMount\(\) Function**](#): This function is invoked right after the component is mounted on the DOM i.e. this function gets invoked once after the `render()` function is executed for the first time

Updation

React is a JS library that helps create Active web pages easily. Now active web pages are specific pages that behave according to their user. For example, let's take the {IDE} webpage, the webpage acts differently with each user. User A might write some code in C in the Light Theme while another User may write Python code in the Dark Theme all at the same time. This dynamic behavior that partially depends upon the user itself makes the webpage an Active webpage. Now how can this be related to Updation? Updation is the phase where the states and props of a component are updated followed by some user events such as clicking, pressing a key on the keyboard, etc. The following are the descriptions of functions that are invoked at different points of the Updation phase.

- [**getDerivedStateFromProps**](#): `getDerivedStateFromProps(props, state)` is a static method that is called just before `render()` method in both mounting and updating phase in React. It takes updated props and the current state as arguments.

```
static getDerivedStateFromProps(props, state) {  
    if(props.name !== state.name){  
        //Change in props  
        return{  
            name: props.name  
        };  
    }  
    return null; // No change to state  
}
```

- [**setState\(\) Function**](#): This is not particularly a Lifecycle function and can be invoked explicitly at any instant. This function is used to update the state of a component.
- [**shouldComponentUpdate\(\) Function**](#): By default, every state or props update re-renders the page but this may not always be the desired outcome, sometimes it is desired that updating the page will not be repainted. The `shouldComponentUpdate()` Function fulfills the requirement by letting React know whether the component's output will be affected by the update or not. `shouldComponentUpdate()` is invoked before rendering an already mounted component when new props or states are being received. If returned false then the subsequent steps of rendering will not be carried out. This function can't be used in the case of [**forceUpdate\(\)**](#). The Function takes the new Props and new State as the arguments and returns whether to re-render or not.
- [**getSnapshotBeforeUpdate\(\) Method**](#): The `getSnapshotBeforeUpdate()` method is invoked just before the DOM is being rendered. It is used to store the previous values of the state after the DOM is updated.

- **componentDidUpdate() Function**: Similarly this function is invoked after the component is rerendered i.e. this function gets invoked once after the render() function is executed after the updation of State or Props.

Unmounting

This is the final phase of the lifecycle of the component which is the phase of unmounting the component from the DOM. The following function is the sole member of this phase.

- **componentWillUnmount() Function**: This function is invoked before the component is finally unmounted from the DOM i.e. this function gets invoked once before the component is removed from the page and this denotes the end of the lifecycle.

We have so far discussed every predefined function there was in the lifecycle of the component, and we have also specified the order of execution of the function.

Implementing the lifecycle methods

Let us now see one final example to finish the article while revising what's discussed above.

First, create a react app and edit your **index.js** file from the src folder.

src index.js:

```
import React from 'react';

class Test extends Component {
  constructor(props) {
    super(props);
    this.state = { hello: "World!" };
  }

  componentDidMount() {
    console.log("componentDidMount()");
  }

  changeState() {
    this.setState({ hello: "Students!" });
  }

  render() {
    return (
      <div>
        <h1>Welcome to React Js  Hello {this.state.hello}</h1>
        <h2>
          <a onClick={this.changeState.bind(this)}>Press
Here!</a>
        </h2>
      </div>);
  }
}
```

```

    shouldComponentUpdate(nextProps, nextState) {
      console.log("shouldComponentUpdate()");
      return true;
    }

    componentDidUpdate() {
      console.log("componentDidUpdate()");
    }
  }

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <Test />
);

```

4.7 React hooks

what is a Hook?

Hooks are functions that let you “hook into” React state and lifecycle features from function components. Hooks don’t work inside classes — they let you use React without classes.

Hook Rules

There are 3 rules for hooks:

- Hooks can only be called inside React function components.
- Hooks can only be called at the top level of a component.
- Hooks cannot be conditional

React `useState` Hook

The React `useState` Hook allows us to track state in a function component.

State generally refers to data or properties that need to be tracking in an application.

To use the `useState` Hook, we first need to `import` it into our component.

Example:

```
import { useState } from "react";
```

Initialize `useState`

We initialize our state by calling `useState` in our function component.

`useState` accepts an initial state and returns two values:

- The current state.
- A function that updates the state.

Example:

```
import { useState } from "react";
```

```
function FavoriteColor() {
```

```
  const [color, setColor] = useState("");
```

```
}
```

Notice that again, we are destructuring the returned values from `useState`.

The first value, `color`, is our current state.

The second value, `setColor`, is the function that is used to update our state.

Example:

`FavoriteColor.js`

```
import { useState } from "react";
```

```
function FavoriteColor() {
```

```
  const [color, setColor] = useState("red");
```

```
  return <h1>My favorite color is {color}!</h1>
```

```
}
```

```
export default FavoriteColor
```

Update State

To update our state, we use our state updater function.

We should never directly update state. Ex: `color = "red"` is not allowed.

Example:

`FavoriteColor.js`

```
import { useState } from "react";
function FavoriteColor() {
  const [color, setColor] = useState("red");

  return (
    <>
      <h1>My favorite color is {color}!</h1>
      <button
        type="button"
        onClick={() => setColor("blue")}
      >Blue</button>
    </>
  )
}
```

What Can State Hold

The `useState` Hook can be used to keep track of strings, numbers, booleans, arrays, objects, and any combination of these!

We could create multiple state Hooks to track individual values.

Example:

Create multiple state Hooks:

```
import { useState } from "react";
function Car() {
  const [brand, setBrand] = useState("Ford");
  const [model, setModel] = useState("Mustang");
  const [year, setYear] = useState("1964");
  const [color, setColor] = useState("red");
  return (
    <>
      <h1>My {brand}</h1>
```

```

    <p>
      It is a {color} {model} from {year}.
    </p>
  </>
)
}
export default Car

```

Example:

Create a single Hook that holds an object:

```

import { useState } from "react";
function Car() {
  const [car, setCar] = useState({
    brand: "Ford",
    model: "Mustang",
    year: "1964",
    color: "red"
  });
  return (
    <>
      <h1>My {brand}</h1>
      <p>
        It is a {color} {model} from {year}.
      </p>
    </>
  )
}
export default Car

```

React useEffect Hooks

The useEffect Hook allows you to perform side effects in your components.

Some examples of side effects are: fetching data, directly updating the DOM, and timers.

useEffect accepts two arguments. The second argument is optional.

useEffect(<function>, <dependency>)

example:

```

import { useState, useEffect } from "react";
function Timer() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    setTimeout(() => {
      setCount((count) => count + 1);
    }, 1000);
  });
  return <h1>I've rendered {count} times!</h1>;
}
export default Timer

```

Example:

Here is an example of a `useEffect` Hook that is dependent on a variable. If the count variable updates, the effect will run again:

```

import { useState, useEffect } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  const [calculation, setCalculation] = useState(0);

  useEffect(() => {
    setCalculation(() => count * 2);
  }, [count]); // <- add the count variable here

  return (
    <>
      <p>Count: {count}</p>
      <button onClick={() => setCount((c) => c + 1)}>+</button>
      <p>Calculation: {calculation}</p>
    </>
  );
}

```

```
}  
export default Counter;
```

React Context

React Context is a way to manage state globally.

It can be used together with the useState Hook to share state between deeply nested components more easily than with useState alone.

The Problem:

State should be held by the highest parent component in the stack that requires access to the state.

To illustrate, we have many nested components. The component at the top and bottom of the stack need access to the state.

To do this without Context, we will need to pass the state as "props" through each nested component. This is called "prop drilling".

Example Component1.js

```
import { useState } from "react";  
function Component1() {  
  const [user, setUser] = useState("Dumbeldore");  
  return (  
    <>  
      <h1>`Hello ${user}!`</h1>  
      <Component2 user={user} />  
    </>  
  );  
}  
  
function Component2({ user }) {  
  return (  
    <>  
      <h1>Component 2</h1>  
      <Component3 user={user} />  
    </>  
  );  
}
```

```
function Component3({ user }) {
  return (
    <>
      <h1>Component 3</h1>
      <Component4 user={user} />
    </>
  );
}
```

```
function Component4({ user }) {
  return (
    <>
      <h1>Component 4</h1>
      <Component5 user={user} />
    </>
  );
}
```

```
function Component5({ user }) {
  return (
    <>
      <h1>Component 5</h1>
      <h2>{`Hello ${user} again!`}</h2>
    </>
  );
}
```

```
export default Component1
```

The Solution

Create Context

To create context, you must Import createContext and initialize it:

```
import { useState, createContext } from "react";
```

```
const UserContext = createContext()
```

Context Provider

Wrap child components in the Context Provider and supply the state value.

```
function Component1() {
```



```

const [user, setUser] = useState("Harry Potter");
return (
  <UserContext.Provider value={user}>
    <h1>`Hello ${user}!`</h1>
    <Component2 user={user} />
  </UserContext.Provider>
);
}

```

Use the useContext Hook

In order to use the Context in a child component, we need to access it using the useContext Hook.

First, include the useContext in the import statement:

```
import { useState, createContext, useContext } from "react";
```

Then you can access the user Context in all components:

```

function Component5() {
  const user = useContext(UserContext);
  return (
    <>
      <h1>Component 5</h1>
      <h2>`Hello ${user} again!`</h2>
    </>
  );
}

```

Example component1.js

```

import { useState, createContext, useContext } from "react";
const UserContext = createContext();

function Component1() {
  const [user, setUser] = useState("Jesse Hall");
  return (

```

```

    <UserContext.Provider value={user}>
      <h1>{`Hello ${user}!`}</h1>
      <Component2 />
    </UserContext.Provider>
  );
}
function Component2() {
  return (
    <>
      <h1>Component 2</h1>
      <Component3 />
    </>
  );
}
function Component3() {
  return (
    <>
      <h1>Component 3</h1>
      <Component4 />
    </>
  );
}

function Component4() {
  return (
    <>
      <h1>Component 4</h1>
      <Component5 />
    </>
  );
}

function Component5() {
  const user = useContext(UserContext);
  return (
    <>
      <h1>Component 5</h1>
      <h2>{`Hello ${user} again!`}</h2>
    </>
  );
}
export default Component1

```

useRef to Reference React Elements

Refs are a special attribute that are available on all React components. They allow us to create a reference to a given element / component when the component mounts.

useRef allows us to easily use React refs. They are helpful (as in the example below) when we want to directly interact with an element, such as to clear its value or focus it, as with an input.

We call useRef (at the top of a component) and attach the returned value to the element's ref attribute to refer to it.

Example:

```
import { useRef } from "react";
function Eg1() {
  const inputElement = useRef();

  const focusInput = () => {
    inputElement.current.focus();
  };
  return (
    <>
      <input type="text" ref={inputElement} />
      <button onClick={focusInput}>Focus Input</button>
    </>
  );
}
export default Eg1
```

React useCallback Hook

The React **useCallback** Hook returns a memoized callback function. Think of memoization as caching a value so that it does not need to be recalculated.

This allows us to isolate resource intensive functions so that they will not automatically run on every render.

The useCallback Hook only runs when one of its dependencies update.

This can improve performance.

The useCallback and useMemo Hooks are similar. The main difference is that useMemo returns a memoized value and useCallback returns a memoized function

Parent.js

```

import React, {useState} from 'react';
import Button from './Button';
import Title from './Title';
import Count from './Count';
function ParentComponent() {
  const [age, setAge] = useState(25);
  const [salary, setSalary] = useState(25000)
  /*  const incrementAge = () => {
    setAge(age + 1);
  }
  const incrementSalary = () => {
    setSalary(salary + 1000);
  }*/
  const incrementAge = useCallback(() => {

    setAge(age + 1);

  }, [age]);
  const incrementSalary = useCallback(() => {

    setSalary(salary + 1000);

  }, [salary]);*/

  return (
    <div>
      <Title/>
      <Count text="age" count={age} />
      <Button handleClick={incrementAge}>Increment my age</Button>
      <Count text="salary" count={salary} />
      <Button handleClick={incrementSalary}>Increment my salary</Button>
    </div>
  );
}
export default ParentComponent;

```

count.js

```

import React from 'react';
function Count(props) {
  console.log("Count rendering");
  return (
    <div>
      {props.text} is {props.count}
    </div>
  );
}

```

```
    );  
  }  
  export default Count;
```

Button Component (Button.js)

```
import React from 'react';  
function Button(props) {  
  console.log(`Button clicked ${props.children}`);  
  return (  
    <div>  
      <button onClick = {props.handleClick}> {props.children} </button>  
    </div>  
  );  
}  
export default Button;
```

Title Component (Title.js)

```
import React from 'react';  
function Title() {  
  console.log ("Title Rendering");  
  return (  
    <div>  
      <h2> useCallBack hook</h2>  
    </div>  
  );  
}  
export default Title;
```

In our application, this re-render is completely unnecessary. So, what could we do to avoid an unnecessary re-render in the component?

In our example, when we increment the age only two things should be re-rendered.

- The count component related to age.
- The button component for incrementing the age.

The other three log statements don't need to be re-rendered. The same is the case with salary. If you increment the salary, title and age components should not be re-rendered. So, how do we optimize this? Well, the answer is `React.Memo()`.

React.memo

React.memo is a Higher Order Component (HOC) that prevents a functional component from being re-rendered if its props or state do not change.

Please keep in mind React.memo() has nothing to do with hooks. It has been a feature since react version 16.6 as class components already allowed you to control re-renders with the use of PureComponent or shouldComponentUpdate.

Let's make use of React.memo() in our example.

In all the three components while exporting, encapsulate/wrap your component in React.memo.

```
import React from 'react';
function Button(props) {
  console.log(`Button clicked ${props.children}`);
  return (
    <div>
      <button onClick = {props.handleClick}> {props.children} </button>
    </div>
  );
}
export default React.memo(Button); //Similarly for other components
```

useReducer Hook

The **useReducer** Hook is the better alternative to the [useState](#) hook and is generally more preferred over the **useState** hook when you have complex state-building logic or when the next state value depends upon its previous value or when the components are needed to be optimized.

The **useReducer** hook takes three arguments including reducer, initial state, and the function to load the initial state lazily.

Syntax:

```
const [state, dispatch] = useReducer(reducer, initialArgs);
```

1. Initial state

The *initial state* is the value the state is initialized with. In our example the initial state is initialed to an object which has a property called `count` initialized with a value of 0.

2. Dispatch function

The `useReducer()` hook returns a special function `dispatch` which dispatches an *action object* to the `reducer` function. In our case, we are calling the `dispatch` function from the `onClick` event handler with the appropriate action object.

3. Action object

An action object is an object that describes how the state should change.

It's considered a good practice to assign the value of a string to a constant if it's used in multiple places.

4. Reducer function

A reducer is a pure function that receives the previous state and an action as arguments and returns the new state based on that action.

Example Counter.js

```
import React, { useReducer } from 'react';
```

```
// Reducer function
```

```
const counterReducer = (state, action) => {
```

```
  switch (action.type) {
```

```
    case 'INCREMENT':
```

```
      return { count: state.count + 1 };
```

```
    case 'DECREMENT':
```

```
      return { count: state.count - 1 };
```

```
    default:
```

```
      return state;
```

```
  }
```

```
};
```

```
const Counter = () => {
```

```
  // Initialize the state using useReducer
```

```
  const [state, dispatch] = useReducer(counterReducer, { count: 0 });
```

```
  // Event handlers for increment and decrement
```

```

const handleIncrement = () => {
  dispatch({ type: 'INCREMENT' });
};

const handleDecrement = () => {
  dispatch({ type: 'DECREMENT' });
};

return (
  <div>
    <h1>Counter</h1>
    <p>Count: {state.count}</p>
    <button onClick={handleIncrement}>Increment</button>
    <button onClick={handleDecrement}>Decrement</button>
  </div>
);
};

export default Counter;

```

Here, we define a counterReducer function. This function takes two parameters:

- state: The current state of the counter.
- action: An object that describes what kind of state change should occur.

Inside the reducer, we use a switch statement to handle different types of actions. In this case, we have two action types:

- 'INCREMENT': It increments the count property in the state by 1.
- 'DECREMENT': It decrements the count property in the state by 1.

If the reducer receives an action type it doesn't recognize, it returns the current state unchanged.

In the Counter component, we use the useReducer hook to initialize our state. We pass in two arguments:

- counterReducer: The reducer function we defined earlier.
- { count: 0 }: The initial state of the counter, which starts at 0.

The useReducer hook returns an array with two elements:

- state: The current state.
- dispatch: A function that allows us to dispatch actions to the reducer to update the state.

Custom Hook:

custom hook useCounter:

1. Import Required Dependencies:

We import the useState hook from React, which allows us to manage state in our custom hook.

2. Define the Custom Hook Function:

```
function useCounter(initialCount = 0) {
  // Initialize the count state using useState
  const [count, setCount] = useState(initialCount);
```

Inside the custom hook function, we start by using the useState hook to create a count state variable and a setCount function to update it. We can also pass an initial count value as an argument to the custom hook. By default, it's set to 0.

3. Define Functions to Manipulate the Count:

```
// Functions to increment and decrement the count
const increment = () => { setCount(count + 1);};
const decrement = () => {setCount(count - 1);};
```

We define two functions, increment and decrement, that allow us to increase or decrease the count state when called. These functions use setCount to update the state.

4. Return Values:

```
// Return the count value and the functions to manipulate it
return { count, increment, decrement };
```

Finally, we return an object that includes the current count value and the increment and decrement functions. This allows any component that uses the useCounter hook to access the count and control its behavior.

5. Export the Custom Hook:

```
export default useCounter;
```

Now, you can use this custom hook in any component to manage a counter:

```

import React from 'react';
import useCounter from './useCounter';

function CounterComponent() {
  const { count, increment, decrement } = useCounter();

  return (
    <div>
      <h1>Counter</h1>
      <p>Count: {count}</p>
      <button onClick={increment}>Increment</button>
      <button onClick={decrement}>Decrement</button>
    </div>
  );
}

```

```
export default CounterComponent;
```

4.8Form in React Js

Controlled Components

In HTML, form elements such as <input>, <textarea>, and <select> typically maintain their own state and update it based on user input. In React, mutable state is typically kept in the state property of components, and only updated with [setState\(\)](#).

We can combine the two by making the React state be the “single source of truth”. Then the React component that renders a form also controls what happens in that form on subsequent user input. An input form element whose value is controlled by React in this way is called a “controlled component”.

For example, if we want to make the previous example log the name when it is submitted, we can write the form as a controlled component

```

import React, {useState} from 'react';
import './App.css';

function Form() {
  const [name , setName] = useState("");
  const [age , setAge] = useState("");
  const [email , setEmail] = useState("");
  const [password , setPassword] = useState("");
  const [confPassword , setConfPassword] = useState("");

  // function to update state of name with
  // value enter by user in form
  const handleChange =(e)=>{

```

```

    setName(e.target.value);
  }
  // function to update state of age with value
  // enter by user in form
  const handleAgeChange =(e)=>{
    setAge(e.target.value);
  }
  // function to update state of email with value
  // enter by user in form
  const handleEmailChange =(e)=>{
    setEmail(e.target.value);
  }
  // function to update state of password with
  // value enter by user in form
  const handlePasswordChange =(e)=>{
    setPassword(e.target.value);
  }
  // function to update state of confirm password
  // with value enter by user in form
  const handleConfPasswordChange =(e)=>{
    setConfPassword(e.target.value);
  }
  // below function will be called when user
  // click on submit button .
  const handleSubmit=(e)=>{
    if(password!==confPassword)
    {
      // if 'password' and 'confirm password'
      // does not match.
      alert("password Not Match");
    }
    else{
      // display alert box with user
      // 'name' and 'email' details .
      alert('A form was submitted with Name :"' + name +
        '" ,Age :"' +age +'" and Email :"' + email + '"');
    }
    e.preventDefault();

  }
  return (
    <div className="App">
      <header>
        <form onSubmit={{e} => {handleSubmit(e)}}>

```

```

    { /*when user submit the form , handleSubmit()
      function will be called . */ }
    <h2> React Js</h2>
    <h3> Sign-up Form </h3>

    <label >
    Name:
    </label><br/>
    <input type="text" value={name} required onChange={(e)=> {handleChange(e)}} /><br/>
    { /*when user write in name input box , handleChange()
      function will be called. */ }
    <label >
    Age:
    </label><br/>
    <input type="text" value={age} required onChange={(e)=> {handleAgeChange(e)}} /><br/>
    { /*when user write in age input box , handleAgeChange()
      function will be called. */ }
    <label>
    Email:
    </label><br/>
    <input type="email" value={email} required onChange={(e)=> {handleEmailChange(e)}} /><br/>
    { /* when user write in email input box , handleEmailChange()
      function will be called. */ }
    <label>
    Password:
    </label><br/>
    <input type="password" value={password} required onChange={(e)=>
    {handlePasswordChange(e)}} /><br/>
    { /* when user write in password input box ,
      handlePasswordChange() function will be called. */ }
    <label>
    Confirm Password:
    </label><br/>
    <input type="password" value={confPassword} required onChange={(e)=>
    {handleConfPasswordChange(e)}} /><br/>
    { /* when user write in confirm password input box ,
      handleConfPasswordChange() function will be called. */ }
    <input type="submit" value="Submit"/>
  </form>
</header>
</div>
);
}

```

```
export default Form;
```

Managing State

As your application grows, it helps to be more intentional about how your state is organized and how the data flows between your components. Redundant or duplicate state is a common source of bugs. In this chapter, you'll learn how to structure your state well, how to keep your state update logic maintainable, and how to share state between distant components.

Reacting to input with state

With React, you won't modify the UI from code directly. For example, you won't write commands like "disable the button", "enable the button", "show the success message", etc. Instead, you will describe the UI you want to see for the different visual states of your component ("initial state", "typing state", "success state"), and then trigger the state changes in response to user input. This is similar to how designers think about UI.

Here is a quiz form built using React. Note how it uses the status state variable to determine whether to enable or disable the submit button, and whether to show the success message instead.

```
import { useState } from 'react';

export default function Form() {
  const [answer, setAnswer] = useState("");
  const [error, setError] = useState(null);
  const [status, setStatus] = useState('typing');

  if (status === 'success') {
    return <h1>That's right!</h1>
  }

  async function handleSubmit(e) {
    e.preventDefault();
    setStatus('submitting');
    try {
      await submitForm(answer);
      setStatus('success');
    } catch (err) {
      setStatus('typing');
      setError(err);
    }
  }
}
```

```

    }
  }

function handleTextareaChange(e) {
  setAnswer(e.target.value);
}
return (
  <>
    <h2>City quiz</h2>
    <p>
      In which city is there a billboard that turns air into drinkable water?
    </p>
    <form onSubmit={handleSubmit}>
      <textarea
        value={answer}
        onChange={handleTextareaChange}
        disabled={status === 'submitting'}
      />
      <br />
      <button disabled={
        answer.length === 0 ||
        status === 'submitting'
      }>
        Submit
      </button>
      {error !== null &&
        <p className="Error">
          {error.message}
        </p>
      }
    </form>
  </>
);
}

```

```

function submitForm(answer) {
  // Pretend it's hitting the network.
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      let shouldError = answer.toLowerCase() !== 'lima'
      if (shouldError) {
        reject(new Error('Good guess but a wrong answer. Try again!'));
      } else {
        resolve();
      }
    }, 1500);
  });
}

```

}